1. Introduction to Security Testing

Definition:
Security testing ensures that applications, APIs, and systems are **free from vulnerabilities**, **resilient against attacks**, and **comply with security standards** (like OWASP, ISO 27001, PCI DSS, etc.).

Key Goals:

- Identify **security loopholes** early.
- Prevent **data breaches** and **unauthorized access**.
- Build **user trust** by ensuring safe digital experiences.

---

2. Security Testing Lifecycle

Stage	Activity	Output
1. Planning	Define scope, test objectives, and security standards.	Security Test Plan
2. Threat Modeling	Identify assets, entry points, and possible threats.	Threat Matrix
3. Vulnerability Detection	Perform manual and automated scans.	Vulnerability Report
4. Exploitation	Try to exploit identified vulnerabilities.	Proof of Concept (PoC)
5. Remediation & Retesting	Validate fixes and verify closure.	Final Test Report

---

3. Common Security Testing Techniques with Examples

A. Input Validation Testing

Objective: Prevent injection attacks by validating all user inputs.
Example:

- Try entering `' OR 1=1 --` in login fields.
- Observe if unauthorized login occurs.

Practice Exercise:

- Use **PortSwigger Lab: SQL Injection UNION attacks**
- Create a dummy login form in HTML + PHP/MySQL and test input sanitization.

Authentication & Session Testing

Objective: Verify login controls, session tokens, and password policies.

Checklist:

- Does the system use **secure cookies (HttpOnly, Secure flag)?**
- Is session token regenerated after login/logout?
- Are weak passwords accepted?

Example:

- Try using default credentials like `admin:admin`, `root:1234`.
- Modify session ID in cookies via Burp Suite to hijack sessions.

Practice Exercise:

- Lab: PortSwigger - Authentication vulnerabilities
- Tool: Burp Suite → Proxy → Intercept login request → Edit session cookie → Replay.

Authorization Testing (Access Control)

Objective: Ensure users cannot access other users' data.

Example:

- User A (ID=101) requests: `/user?id=101`
- Modify to `/user?id=102` → Access User B data?

Expected: 403 Forbidden.

Practice Exercise:

- Lab: PortSwigger - Broken Access Control

Security Misconfiguration

Objective: Detect unpatched software, open ports, default settings.

Example:

- Check HTTP headers like `X-Powered-By: PHP/7.0.0` → Outdated.

Use Nmap:

```
nmap -sV targetsite.com
```

-
- Output showing old Apache version = Vulnerable system.

Practice Exercise:

- Lab: PortSwigger - Security Misconfiguration
- Tool: OWASP ZAP → Passive Scan → Check server headers.

Sensitive Data Exposure

Objective: Ensure data is encrypted during transmission and storage.

Example:

- Observe login form → Is it using **HTTPS**?

- Check if passwords are stored as plaintext in DB dump.

Practice Exercise:

- Use **Burp Suite** → **Repeater** to intercept HTTPS → check for unencrypted credentials.
- Lab: PortSwigger - Sensitive Data Exposure

Cross-Site Scripting (XSS)

Objective: Identify where scripts can be injected into web pages.

Example:

- Input field: `<script>alert('XSS')</script>`
- If alert pops → app is vulnerable.

Practice Exercise:

- Lab: PortSwigger - Reflected XSS
- Try Stored XSS by adding malicious comments in forums.

Cross-Site Request Forgery (CSRF)

Objective: Prevent unauthorized actions on behalf of authenticated users.

Example:

- Craft a fake form that triggers `/updateEmail?user=101&email=hacker@mail.com`
- Send link to logged-in user; if clicked, account changes happen silently.

Practice Exercise:

- Lab: PortSwigger - CSRF

- Use Burp Suite → Generate CSRF PoC.

API Security Testing

Objective: Ensure APIs have proper authentication, validation, and rate-limiting.

Checklist:

- No hardcoded API keys.
- Enforce authentication for all endpoints.
- Input validation for parameters.

Example:

- API: `/api/getUser?id=1`
- Try changing ID → Access others' data?

Practice Exercise:

- Tool: Postman or OWASP ZAP API mode.
- Test Case: Add invalid/malicious JSON payloads.
- Reference: OWASP API Security Top 10

File Upload Testing

Objective: Prevent malicious files from being uploaded.

Example:

- Upload `.php` file disguised as `.jpg`
- Observe if code executes on the server.

Practice Exercise:

- Lab: PortSwigger - File Upload Vulnerabilities
- Verify file type, size, and extension validations.

Server-Side Request Forgery (SSRF)

Objective: Prevent internal network access through crafted URLs.

Example:

- Parameter: `/fetch?url=http://internal-system/admin`
- Modify to internal IPs (e.g., `http://127.0.0.1`)

Practice Exercise:

- Lab: PortSwigger - SSRF
- Check if internal metadata (AWS, Azure) is exposed.

---

4. Security Testing Tools (Expanded Practice Table)

Category	Tool	Practice Activity
Web Proxy	**Burp Suite (Community)**	Intercept & modify requests/responses
Scanner	**OWASP ZAP**	Run automated OWASP Top 10 scans
Network	**Nmap**	Map open ports & services
Password	**Hydra / John the Ripper**	Perform brute-force test on login
Code Analysis	**SonarQube / Bandit**	Detect insecure functions in code
Vulnerability Assessment	**Nessus / OpenVAS**	Generate vulnerability reports
Exploitation	**Metasploit**	Simulate real-world exploit scenarios

---

5. Practice Environment Setup

Create a **safe lab environment** using:

- **DVWA (Damn Vulnerable Web Application):**
  👉 https://dvwa.co.uk/
- **bWAPP (Buggy Web Application):**
  👉 https://sourceforge.net/projects/bwapp/
- **OWASP Juice Shop:**
  👉 https://owasp.org/www-project-juice-shop/
- **PortSwigger Web Security Academy Labs:**
  👉 https://portswigger.net/web-security

🧩 *Tip:* Always run in isolated VMs using **VirtualBox** or **Docker** to avoid risk.

---

6. Real-World QA Security Testing Checklist

Area	Checkpoints
Login	Enforce strong passwords, lock after failed attempts
Session	Secure, unique, time-bound tokens
Data	Encryption in transit & at rest
APIs	Authentication + Rate limiting
Error Handling	No stack trace in production
Logging	Enable monitoring & alerting for suspicious activities
File Uploads	Validate extensions, disable executable uploads

---

7. Weekly Practice Plan (Mentorship-Ready)

Week	Focus	Assignments
Week 1	OWASP Top 10 Overview	Research and create a summary doc
Week 2	Burp Suite Basics	Complete 3 PortSwigger Labs

Week 3	Authentication & Access Control	Perform Broken Auth test on DVWA
Week 4	API Testing & Security	Secure API test with Postman & ZAP
Week 5	Threat Modeling & Reporting	Create a vulnerability report for a mock project

---

8. Reporting & Communication

Example Vulnerability Report:

Field	Details
Title	SQL Injection in Login Page
Severity	High
CVSS Score	8.8
Description	Login field vulnerable to SQL injection
Steps to Reproduce	Enter `' OR '1'='1` in username
Impact	Unauthorized access
Remediation	Use parameterized queries, input validation
Status	Open / Fixed

---

9. Reference Links

- 🌐 **OWASP Testing Guide:** https://owasp.org/www-project-web-security-testing-guide/
- 🧠 **PortSwigger Academy:** https://portswigger.net/web-security
- 🧩 **TryHackMe Labs:** https://tryhackme.com/
- 🧱 **HackTheBox:** https://www.hackthebox.com/
- 🛠️ **MITRE ATT&CK Framework:** https://attack.mitre.org/

- 🧾 **CWE Database:** https://cwe.mitre.org/
